SV Interface Demo

Release 1.0.0

HDL Team

Nov 01, 2026

Design

1 SV Interface Demo — Overview **1**
- 1.1 Instantiation Tree . . . 1
- 1.2 Modules . . . 1

2 top **3**
- 2.1 top — Entity . . . 3
  - 2.1.1 Annotated Source . . . 3
- 2.2 top — Block Diagram . . . 3
  - 2.2.1 Ports . . . 4
  - 2.2.2 Interfaces . . . 4
- 2.3 top — State Machine . . . 5
- 2.4 top — Timing Diagrams . . . 5
- 2.5 top — Clock Domain Analysis . . . 5
- 2.6 top — Reset Domain Analysis . . . 5
- 2.7 Submodules . . . 5
  - 2.7.1 bridge . . . 5
    - 2.7.1.1 bridge — Entity . . . 5
    - 2.7.1.2 bridge — Block Diagram . . . 6
    - 2.7.1.3 bridge — State Machine . . . 9
    - 2.7.1.4 bridge — Timing Diagrams . . . 9
    - 2.7.1.5 bridge — Clock Domain Analysis . . . 9
    - 2.7.1.6 bridge — Reset Domain Analysis . . . 9

3 Indices **11**

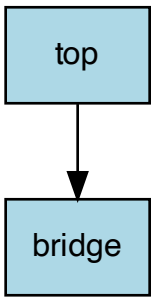
Chapter 1

SV Interface Demo — Overview

This project contains 2 VHDL/SV modules.

Top-level module: *top*

1.1 Instantiation Tree



1.2 Modules

Module	Source File	Description	Instantiated by	Instantiates
bridge	`bridge.sv`	Bridges a register write port to a stream. Takes AXI4-Lite writes and forwards the data as an AXI4-Stream.	`top`	—
top	`top.sv`	Top level.	*(top)*	`bridge`

Chapter 2

top

2.1 top — Entity

Source file: `src/top.sv`.

Top level.

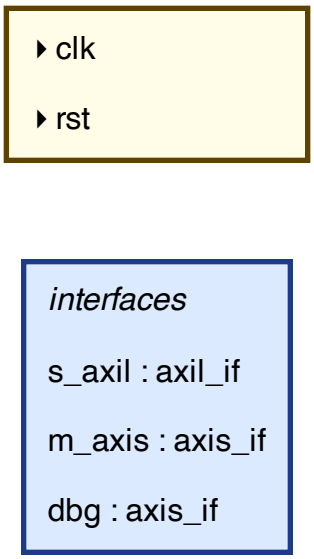
2.1.1 Annotated Source

Listing 1: src/top.sv

```
// Top level.
module top (
    input  logic clk,
    input  logic rst
);
    axil_if s_axil (.clk(clk));
    axis_if m_axis (.clk(clk));
    axis_if dbg    (.clk(clk));
    bridge u_bridge (.clk(clk), .rst(rst), .s_axil(s_axil), .m_axis(m_axis), .s_
↪dbg(dbg));
endmodule
```

2.2 top — Block Diagram

Auto-extracted from `top.sv`.



2.2.1 Ports

Port	Direction	Type	Description
`clk`	`in`	`logic`	—
`rst`	`in`	`logic`	—

2.2.2 Interfaces

`axil_if` — instantiated here as `s_axil`.

Signal	Type	slave	master	Description
`awaddr`	`logic[31:0]`	in	out	—
`awvalid`	`logic`	in	out	—
`awready`	`logic`	out	in	—
`wdata`	`logic[31:0]`	in	out	—

`axis_if` — instantiated here as `m_axis`, `dbg`.

Signal	Type	master	slave	Description
`tdata`	`logic[7:0]`	out	in	Payload byte.
`tvalid`	`logic`	out	in	Handshake.
`tready`	`logic`	in	out	Handshake.
`tlast`	`logic`	out	in	—

2.3 top — State Machine

No FSM detected in `top.sv`.

This module is either a structural wrapper (no combinational state logic) or uses a coding style not yet supported by the FSM extractor.

2.4 top — Timing Diagrams

All timing diagrams extracted from `top.sv`. Diagrams are sourced from `.. wavedrom::` blocks in the VHDL source comments above each process.

No wavedrom diagrams found in process comments.

Add `.. wavedrom::` blocks in the VHDL source above each process to have them appear here automatically.

2.5 top — Clock Domain Analysis

Auto-extracted from `top.sv`.

Note

No clocked processes detected. This module appears to be purely combinational.

2.6 top — Reset Domain Analysis

Auto-extracted from `top.sv`.

Note

No clocked processes detected. This module appears to be purely combinational.

2.7 Submodules

2.7.1 bridge

2.7.1.1 bridge — Entity

Source file: `src/bridge.sv`.

Bridges a register write port to a stream.

Takes AXI4-Lite writes and forwards the data as an AXI4-Stream.

Annotated Source

Listing 2: src/bridge.sv

```
1 // Bridges a register write port to a stream.
2 //
3 // Takes AXI4-Lite writes and forwards the data as an AXI4-Stream.
```

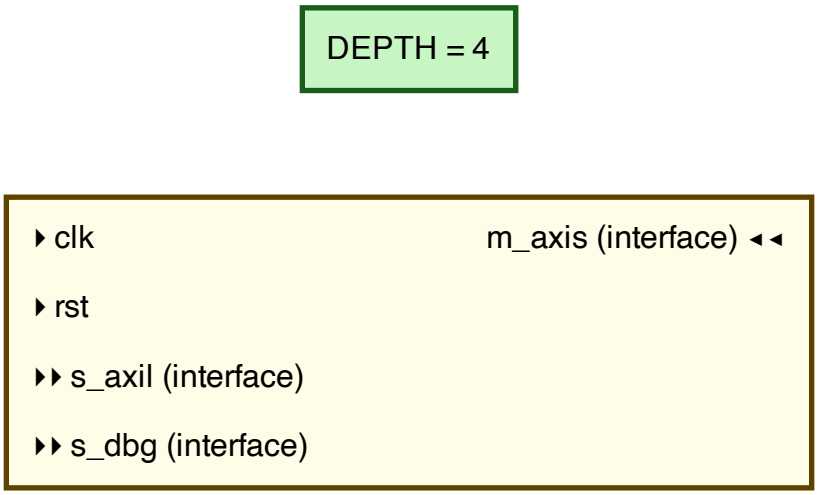
(continues on next page)

(continued from previous page)

```
module bridge #(
    parameter int DEPTH = 4
)(
    // System clock.
    input  logic       clk,
    // Synchronous reset.
    input  logic       rst,
    // Register write interface.
    axil_if.slave      s_axil,
    // Outgoing byte stream.
    axis_if.master     m_axis,
    axis_if            s_dbg       // Debug tap, no modport.
);
    assign s_axil.awready = 1'b1;
    assign m_axis.tdata   = s_axil.wdata[7:0];
    assign m_axis.tvalid  = s_axil.awvalid;
    assign m_axis.tlast   = 1'b0;
endmodule
```

2.7.1.2 bridge — Block Diagram

Auto-extracted from `bridge.sv`.



Ports

Port	Direction	Type	Description
clk	in	logic	System clock.
rst	in	logic	Synchronous reset.
s_axil	slave	axil_if	Register write interface.
m_axis	master	axis_if	Outgoing byte stream.
s_dbg	—	axis_if	Debug tap, no modport.

Interfaces

axil_if — used by s_axil.

Signal	Type	slave	Description
awaddr	logic[31:0]	in	—
awvalid	logic	in	—
awready	logic	out	—
wdata	logic[31:0]	in	—

axis_if — used by m_axis, s_dbg.

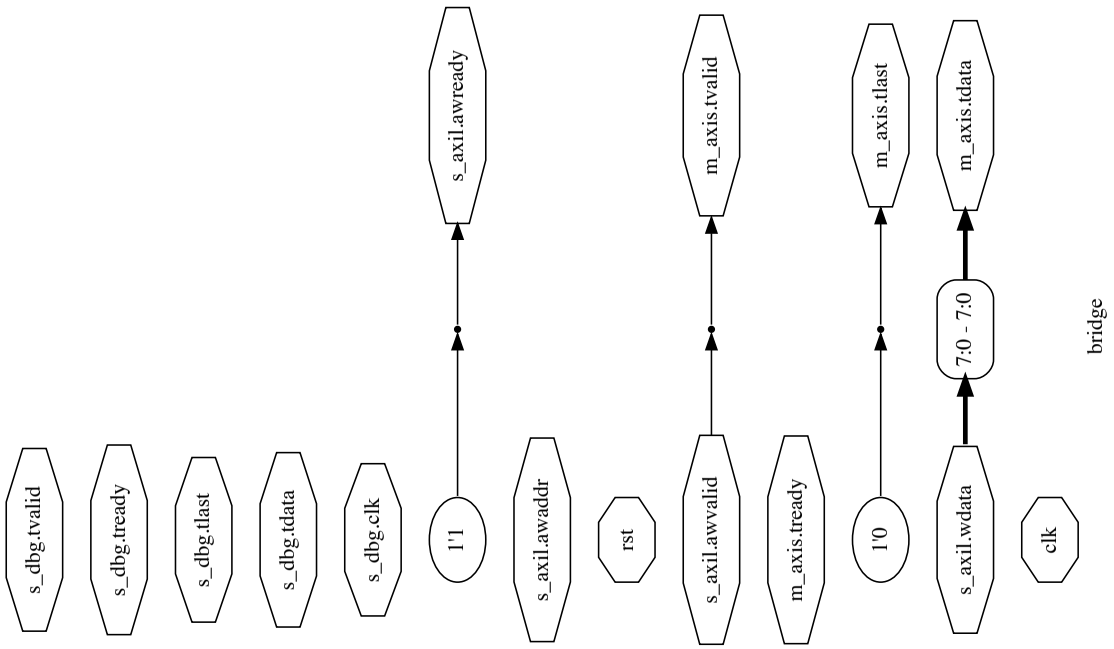
Signal	Type	master	Description
tdata	logic[7:0]	out	Payload byte.
tvalid	logic	out	Handshake.
tready	logic	in	Handshake.
tlast	logic	out	—

Parameters

Name	Type	Default	Description
DEPTH	int	4	—

RTL Detail

Gate-level netlist generated by yosys and rendered with netlistsvg.


s_dbg.tvalid
s_dbg.tready
s_dbg.tlast
s_dbg.tdata
s_dbg.clk
1'1
s_axil.awready
s_axil.awaddr
rst
s_axil.awvalid
m_axis.tready
m_axis.tvalid
1'0
m_axis.tlast
s_axil.wdata
7:0 - 7:0
m_axis.tdata
clk
bridge

2.7.1.3 bridge — State Machine

No FSM detected in `bridge.sv`.

This module is either a structural wrapper (no combinational state logic) or uses a coding style not yet supported by the FSM extractor.

2.7.1.4 bridge — Timing Diagrams

All timing diagrams extracted from `bridge.sv`. Diagrams are sourced from `.. wavedrom::` blocks in the VHDL source comments above each process.

No wavedrom diagrams found in process comments.

Add `.. wavedrom::` blocks in the VHDL source above each process to have them appear here automatically.

2.7.1.5 bridge — Clock Domain Analysis

Auto-extracted from `bridge.sv`.

Note

No clocked processes detected. This module appears to be purely combinational.

2.7.1.6 bridge — Reset Domain Analysis

Auto-extracted from `bridge.sv`.

Note

No clocked processes detected. This module appears to be purely combinational.

Chapter 3

Indices

- genindex
- search